

Day 3 - CSS Basics: Selectors, the Cascade, Specificity, the Box Model

Day 3 — CSS Basics: Selectors, the Cascade, Specificity, the Box Model

Objectives

- Understand what a stylesheet is and the three ways to apply CSS to a page, and why one is strongly preferred
- Write selectors that target exactly the elements you want to style
- Understand the cascade and specificity — how the browser decides which rule wins when several could apply to the same element
- Understand the box model in real depth — arguably the single most important idea in all of CSS

What is CSS, mechanically?

CSS stands for Cascading Style Sheets. A **rule** consists of a **selector** (which elements should this apply to?) followed by a block of **declarations** in curly braces (what should change about them?):

```
p {  
  color: blue;  
  font-size: 18px;  
}
```

Read this as: "select every `<p>` element, and set its text color to blue and its font size to 18 pixels." Each individual line inside the braces — `color: blue;` — is one **declaration**, made of a **property** (`color`) and a **value** (`blue`), separated by a colon and ended with a semicolon.

Three ways to apply CSS — and why external is the one you should actually use

1. Inline — directly on one specific element, via a `style` attribute:

```
<p style="color: blue; font-size: 18px;">Some text</p>
```

2. Internal — inside a `<style>` block in the page's `<head>`:

```
<head>  
  <style>  
    p {  
      color: blue;  
    }  
  </style>  
</head>
```

3. External — in a completely separate `.css` file, linked from the HTML's `<head>`:

```

<head>
  <link rel="stylesheet" href="styles.css">
</head>

/* styles.css */
p {
  color: blue;
}

```

Always prefer external stylesheets for any real project, and that's what you'll do starting today's exercise. Why: it cleanly separates structure (HTML) from appearance (CSS), exactly matching the "wooden frame vs. paint and furniture" distinction from Day 0; it lets you apply the *exact same* styling to every single page of a multi-page site by linking the same one `styles.css` file from each page, instead of retyping styles everywhere; and it makes your HTML dramatically easier to read, since it isn't cluttered with styling details mixed into the content. Inline styles are occasionally useful for a genuinely one-off, single-element tweak, but relying on them as your main approach is considered poor practice in any real project — you'll be marked down for it, informally, by any reviewer at a job.

CSS comments look like this: `/* a comment */` — different syntax from HTML's `<!-- -->`, so don't mix them up.

Selectors — precisely choosing what to style

```

p { }                /* every <p> element */
.highlight { }       /* every element with class="highlight" (note the leading DOT) */
#main-title { }      /* the ONE element with id="main-title" (note the leading HASH/pound sign)
nav a { }            /* every <a> that is anywhere INSIDE a <nav> -- a "descendant" selector
p.intro { }          /* every <p> that ALSO has class="intro" -- no space, combining two
h1, h2, h3 { }        /* every h1 AND every h2 AND every h3 -- a comma separates a shared

```

Recall from Day 1: `class` can label many elements at once (so `.highlight` might match several elements across the page), while `id` must be unique to exactly one element per page (so `#main-title` matches, at most, exactly one). As a strong rule of thumb for the next two weeks: **reach for classes to style groups of similar elements** (this is what you'll do the vast majority of the time), **and reserve id selectors mainly for the rare cases where you genuinely need to target one single, unique element** (or, more often, for `href="#some-id"` in-page jump links from Day 1, and for JavaScript to hook into, which is outside this course's scope).

The cascade and specificity — how the browser resolves conflicts

The "C" in CSS stands for **Cascading** — meaning multiple rules can apply to the very same element simultaneously, and CSS needs a consistent, predictable set of rules for deciding which one actually wins when they conflict. This is one of the ideas beginners find most confusing at first, so let's build it up carefully.

Consider:

```

p { color: black; }
.warning { color: red; }
#special-message { color: green; }

<p id="special-message" class="warning">What color is this text?</p>

```

All three rules technically match this one `<p>` — so which color wins? CSS resolves this using **specificity**: a scoring system where more specific selectors beat more general ones, regardless of the order they're written in. The scoring, roughly, from lowest to highest specificity:

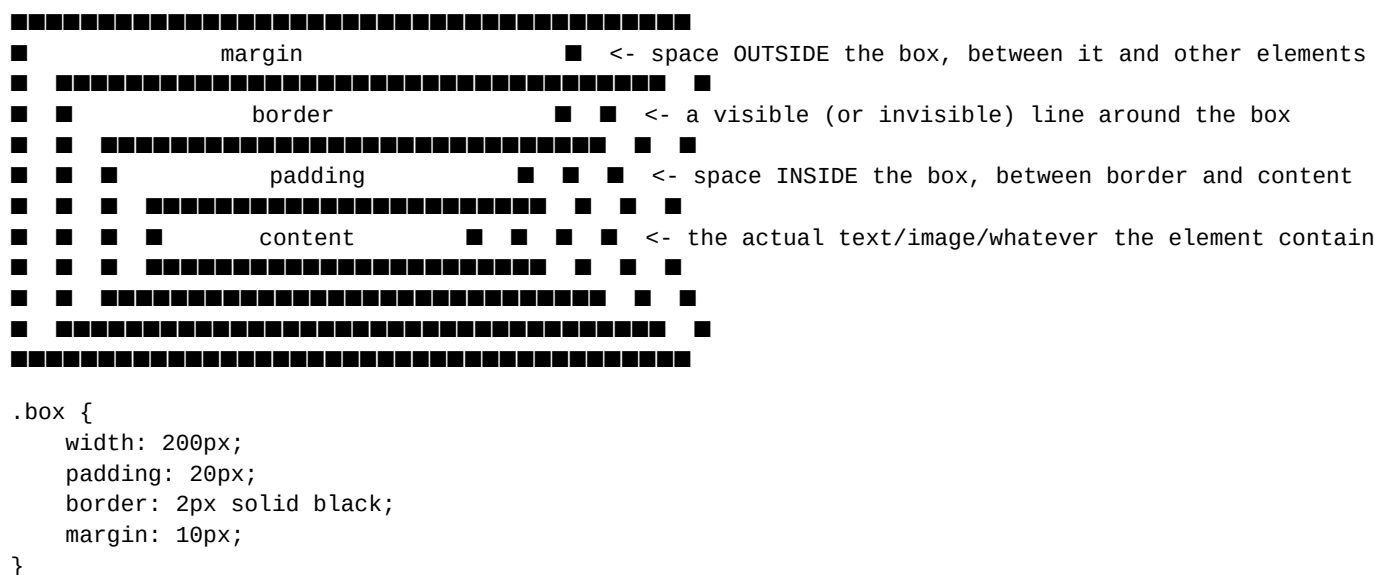
- 1 Element selectors (`p`, `div`, `a`) — the least specific.
- 2 Class selectors (`.warning`), and a couple of similar selector types you'll meet later (attribute selectors, pseudo-classes — Day 9).
- 3 ID selectors (`#special-message`) — more specific than any number of classes.
- 4 Inline `style="..."` attributes — beats essentially everything above.
- 5 `!important` appended to a declaration — an escape hatch that overrides normal specificity entirely; **avoid this in real code** — it's a sign of a specificity problem you should actually fix (often by simplifying your selectors, a topic you'll dig into on Day 11), not paper over, since once you start using `!important`, you eventually need another `!important` to override that one, and the cascade rapidly becomes unmanageable.

In the example above, the ID selector (`#special-message { color: green; }`) wins, regardless of the fact that it's written first in the file — **specificity, not source order, decides the winner when selectors have different specificity**. Source order only matters as a **tiebreaker**, when two rules have exactly equal specificity — in that case, whichever rule appears later in the CSS wins:

```
p { color: black; }
p { color: purple; } /* this one wins -- same specificity as above, but written LATER */
```

The box model — the single most important idea in all of CSS

Every single element on a web page is rendered as a rectangular box, whether it's a paragraph, a heading, an image, or an entire section — and every one of those boxes is built from exactly four layers, nested inside each other like rings:



Each of the four layers can be controlled individually per side, or all at once:

```

.box {
  margin-top: 10px;
  margin-right: 20px;
  margin-bottom: 10px;
  margin-left: 20px;

  /* shorthand for the exact same thing, in TOP - RIGHT - BOTTOM - LEFT order (clockwise, starting top) */
  margin: 10px 20px 10px 20px;

  /* if you only give two values, the first is used for top/bottom, the second for left/right */
  margin: 10px 20px;

  /* one value applies to all four sides equally */
  margin: 10px;
}

```

`padding` works identically to `margin`, just for the inner layer instead of the outer one. `border` needs three pieces of information (width, style, and color): `border: 2px solid black;`.

The genuinely crucial gotcha: what does `width: 200px` actually measure?

By default, in CSS's traditional box model, `width` and `height` only measure the **content** area — padding and border then get added *on top of* that, making the element's actual, total rendered size larger than the `width` you specified:

```

.box {
  width: 200px;
  padding: 20px;
  border: 2px solid black;
}
/* actual total rendered width = 200 (content) + 20 + 20 (left+right padding) + 2 + 2 (left+right border)

```

This trips up nearly every beginner at least once — you set `width: 200px`, add some padding to give the content breathing room, and suddenly the box is wider than you expected and doesn't line up the way you planned. The fix, universally applied in essentially every modern real-world project, is one single declaration:

```

* {
  box-sizing: border-box;
}

```

`box-sizing: border-box` changes what `width` measures: it now means "this box's *total* width, padding and border included" — so if you set `width: 200px` with `box-sizing: border-box`, the box is genuinely, exactly 200px wide overall, no matter how much padding or border you add (the browser shrinks the available content area to compensate, instead of growing the total box). The `*` selector matches literally every single element on the page, which is why this one rule, placed once near the top of your stylesheet, fixes this behavior everywhere at once. **You'll add this exact rule to the very top of every stylesheet you write for the rest of this course, and almost certainly for the rest of your career** — it's such a universal, sensible default that its *absence*, not its presence, is what would need explaining in a real project.

Exercises

Open `starter.html` and `starter.css` together, follow the `<!-- TODO -->` `/* TODO */` markers in both files, and check your work by reloading in the browser and comparing against `CHECKLIST.md`.